



LI.FI

LI.FI Security Review

LiFiIntentEscrowFacet.sol(v1.1.2)

Security Researcher

Sujith Somraaj (somraajsujith@gmail.com)

Report prepared by: Sujith Somraaj

June 9, 2026

Contents

1	About Researcher	2
2	Disclaimer	2
3	Scope	2
4	Risk classification	2
4.1	Impact	2
4.2	Likelihood	3
4.3	Action required for severity levels	3
5	Executive Summary	3
6	Findings	4
6.1	Informational	4
6.1.1	minAmount doubles as the swap floor and the output-scaling denominator	4
6.1.2	Linear outputAmount scaling assumes a size-invariant input+output ratio	4

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over \$170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast and Angles

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

- src/Facets/LiFiIntentEscrowFacet.sol(v1.1.2)

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

- High** leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium** global losses <10% or losses to only a subset of users, but still unacceptable.
- Low** losses will be annoying but bearable — applies to things like grieving attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

- High** almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium** only conditionally possible or incentivized, but still relatively likely
- Low** requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

- Critical** Must fix as soon as possible (if already deployed)
- High** Must fix (before deployment if not already deployed)
- Medium** Should fix
- Low** Could fix

5 Executive Summary

Over the course of 2 hours in total, [LI.FI](#) engaged with the [researcher](#) to audit the contracts described in section 3 of this document ("scope").

In this period of time a total of 2 issues were found.

Project Summary	
Project Name	LI.FI
Repository	lifinance/contracts
Commit	828dc476
Audit Timeline	June 08, 2026
Methods	Manual Review
Documentation	High
Test Coverage	High

Issues Found	
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	0
Gas Optimizations	0
Informational	2
Total Issues	2

6 Findings

6.1 Informational

6.1.1 minAmount doubles as the swap floor and the output-scaling denominator

Context: [LiFiIntentEscrowFacet.sol#L146-L159](#)

Description: On the swap entrypoint, `_bridgeData.minAmount` serves two roles at once: it is the slippage floor passed to `_depositAndSwap`, and it is the denominator used to scale the committed output:

```
uint256 swapMinAmountOut = _bridgeData.minAmount;
uint256 swapOutcome = _depositAndSwap(... swapMinAmountOut ...);
uint256 effectiveOutputAmount =
    (swapOutcome * _lifiIntentData.outputAmount) / swapMinAmountOut;
```

Because the same value gates the swap and prices the intent, the two concerns can no longer be tuned independently. Setting `minAmount` below the true worst-case output the backend quoted `outputAmount` against e.g. a loose floor to keep the swap from reverting shrinks the denominator and inflates `effectiveOutputAmount`, committing an output the backend never quoted.

Recommendation: Document the invariant in the `outputAmount / function NatSpec`: on the swap path, `_bridgeData.minAmount` must equal the exact worst-case swap output that `outputAmount` was quoted against, since it is both the swap slippage floor and the output-scaling denominator.

LI.FI: Fixed in [08d4a58](#)

Researcher: Verified fix.

6.1.2 Linear outputAmount scaling assumes a size-invariant input→output ratio

Context: [LiFiIntentEscrowFacet.sol#L158-L160](#)

Description: On the swap entrypoint, positive slippage is no longer refunded; instead the realized swap output funds the intent and `outputAmount` is scaled linearly off the backend's quoted ratio:

```
uint256 effectiveOutputAmount =
    (swapOutcome * _lifiIntentData.outputAmount) / swapMinAmountOut;
```

This holds the intent's input → output exchange rate constant at the quoted `outputAmount / minAmount`, regardless of the realized input size. That assumption is exact for a purely proportional rate, but a solver's fillable rate is not strictly proportional and it also reflects flat/tiered components (gas, fixed fees) and liquidity-dependent pricing that don't scale linearly with size. When the realized input exceeds the quoted `minAmount`, the scaled `effectiveOutputAmount` may diverge from what a fresh quote at the larger size would yield and could result in bridging failures in some cases.

Recommendation: This is acceptable as designed. Make the assumption explicit so integrators understand the trade-off, by documenting in the `outputAmount / function NatSpec`.

LI.FI: Fixed in [08d4a58](#)

Researcher: Verified fix.